# คู่มือนำอ่านเปเปอร์สัมมนา: Collatz-Weyl PRNG & NIST Randomness Testing

## ออกแบบตัวสร้างเลขสุ่มเทียมความเร็วสูงและการทดสอบสากล (2023) สำหรับนิสิตปริญญาตรี

อาจารย์ ดร.กิตติพงศ์ ทรัพย์วัฒนาชัย
ภาควิชาวิทยาการคำนวณและเทคโนโลยีดิจิทัล คณะศิลปศาสตร์และวิทยาศาสตร์ มหาวิทยาลัยเกษตรศาสตร์ กำแพงแสน

2026-09-30

## Table of contents

1 1. ทำไมต้องเป็นเปเปอร์นี้? (เปเปอร์นี้มีดีตรงไหน) ..... 1
2 2. ก้าวข้าม “สัมมนาแบบท่องจำ”: อ่านแล้วเด็กจะได้อะไรจริง ๆ ..... 2
3 3. ความเหมาะสมกับคณิตศาสตร์ประยุกต์และวิทยาการคอมพิวเตอร์ ..... 2
4 4. สำหรับเด็กที่ไม่เคยอ่านเปเปอร์มาก่อนเลย: เริ่มต้นอย่างไร? ..... 2
5 5. จุดบกพร่องและคำถามเปิด: โอกาสทองในการทำวิจัย ..... 2
6 6. แนวทางการต่อยอดสู่เล่มโครงงานปริญญาตรี 5 บท (01417499) ..... 3
7 7. บันไดฝึกฝน 4 ขั้นสำหรับนิสิต ..... 3

**เป้าหมายของคู่มือฉบับนี้**

คู่มือนี้จัดทำขึ้นสำหรับนิสิตสัมมนาคณิตศาสตร์ประยุกต์และวิทยาการคอมพิวเตอร์ ที่สนใจ **วิทยาการรหัสลับและการคำนวณสมรรถนะสูง** (High-Performance Computing & Cryptography):
**Tomasz R. Dziala, “Collatz-Weyl PRNG: A High-Performance Pseudorandom Number Generator,” arXiv:2312.17043, 2023.**

---

# 1 1. ทำไมต้องเป็นเปเปอร์นี้? (เปเปอร์นี้มีดีตรงไหน)

1. **แก้ปัญหาคลาสสิกด้วยคณิตศาสตร์ที่หลักแหลม:** ฟังก์ชันคอลลัทซ์ทั่วไปติดกับดัก $4 \to 2 \to 1$ เสมอ Dziala นำ **ลำดับของไวล์ (Weyl Sequence)** เข้ามาบวกกวนบิต ทำให้การันตีคาบยาวมหาศาล $2^{64}$ ได้ 100%
2. **ขั้นตอนวิธีสั้นและเร็วระดับสายฟ้า:** โค้ดทั้งหมดใช้เพียงการเลื่อนบิต (Shift) และ XOR ระดับ 64-bit สามารถปั่นตัวเลขสุ่มได้เร็วกว่าหลาย GB ต่อวินาที

3. **ทดสอบด้วยมาตรฐานระดับโลกของรัฐบาลสหรัฐฯ: นำบิตที่ได้ไปรันผ่านชุดทดสอบ NIST SP 800-22 วัดความสุ่มอย่างเป็นรูปธรรม**

---

# 2 2. ก้าวข้าม "สัมมนาแบบท่องจำ": อ่านแล้วเด็กจะได้อะไรจริง ๆ

1. **ได้ฝึกเขียนโค้ดระดับบิต (Bitwise Mastery): นิสิตจะได้ฝึกการเลื่อนบิต** >>, << **และการ** XOR ^ **ใน** Python/C
2. **เข้าใจการวัดผลความเร็ว (Throughput Benchmark): วัดประสิทธิภาพการปั่นข้อมูลเปรียบเทียบกับ** Mersenne Twister **และ** Xorshift
3. **การแปลผลสถิติ $p$-value ของ NIST: เข้าใจการทดสอบ** Frequency Test, Runs Test, **และ** Block Frequency Test

---

# 3 3. ความเหมาะสมกับคณิตศาสตร์ประยุกต์และวิทยาการคอมพิวเตอร์

| หัวข้อในเปเปอร์ | วิชาที่เกี่ยวข้องในหลักสูตร | ประโยชน์ที่นิสิตได้รับ |
|---|---|---|
| **Weyl Sequence Modulo $2^{64}$** | ทฤษฎีจำนวนเชิงคำนวณ | ศึกษาการแจกแจงแบบสม่ำเสมอบนกรุปจำกัด $\mathbb{Z}/2^{64}\mathbb{Z}$ |
| **Bitwise Operations** | สถาปัตยกรรมคอมพิวเตอร์ | การใช้ตรรกะระดับบิตเพื่อความเร็วสูงสุดในการประมวลผล |
| **NIST SP 800-22 Suite** | สถิติประยุกต์และความปลอดภัย | การทดสอบสมมติฐานสถิติความสุ่มระดับสากล |

---

# 4 4. สำหรับเด็กที่ไม่เคยอ่านเปเปอร์มาก่อนเลย: เริ่มต้นอย่างไร?

1. **รอบที่ 1: อ่าน** Abstract **และ** Section 1 **เพื่อทำความเข้าใจว่าทำไม** Collatz **ปกติถึงสุ่มไม่ได้ และ** Weyl Sequence **มาช่วยได้อย่างไร**
2. **รอบที่ 2: ไล่โค้ดใน** Section 2 **ของเปเปอร์ ดูการทำงานของตัวแปร $w$ และ $x$**
3. **รอบที่ 3: เขียนโค้ดทดลองสุ่มตัวเลข** 1,000,000 **ตัวแรกใน** Python **และพล็อตฮิสโตแกรม**

---

# 5 5. จุดบกพร่องและคำถามเปิด: โอกาสทองในการทำวิจัย

- **ถึงแม้จะผ่านการทดสอบ** NIST **แต่** Collatz-Weyl PRNG **เป็นขั้นตอนวิธีเชิงเส้นเป็นช่วง** (Piecewise Linear) **อาจมีจุดอ่อนต่อการโจมตีด้วย** SAT Solvers
- **นิสิตสามารถนำไปทดสอบกับชุดทดสอบความสุ่มที่เข้มงวดกว่า เช่น PractRand หรือ TestU01 (BigCrush)**

---

# 6 6. แนวทางการต่อยอดสู่เล่มโครงงานปริญญาตรี 5 บท (01417499)

- **Track A (Cryptographic PRNG Hardening): เสริมความแข็งแกร่งให้กับขั้นตอนวิธีโดยเพิ่มฟังก์ชันไม่เชิงเส้น** (Non-linear Permutation / S-box)
- **Track B (Parallel GPU PRNG): พัฒนาตัวสร้างเลขสุ่มบน** GPU **ผ่าน** CUDA **หรือ** OpenCL **สำหรับการจำลอง** Monte Carlo **ขนาดใหญ่**

---

# 7 7. บันไดฝึกฝน 4 ขั้นสำหรับนิสิต

1. **ภารกิจที่ 1: เขียนคลาส** `CollatzWeylPRNG` **ใน** Python **และตรวจสอบการทำงาน**
2. **ภารกิจที่ 2: ปั่นข้อมูลไบนารีขนาด** 10 MB **บันทึกลงไฟล์**
3. **ภารกิจที่ 3: ติดตั้งและรันชุดทดสอบ** NIST SP 800-22 **กับไฟล์ข้อมูล**
4. **ภารกิจที่ 4: วัดเวลาและความเร็ว** (MB/s) **เปรียบเทียบกับ** `random` **มาตรฐานของ** Python